EduGenie: Google Gemini Powered Learning Assistant

Project Description

EduGenie is a lightweight AI-powered educational assistant that simplifies learning through generative AI. Designed for students of all academic levels, EduGenie enables users to:

- Ask questions and receive smart, concise answers
- Understand complex concepts through simplified explanations
- Generate quizzes from topics or text
- Receive personalized learning recommendations
- Summarize large educational passages

Built with **FastAPI** for the backend and a simple **HTML+CSS frontend**, EduGenie leverages lightweight and cloud-based AI models for local efficiency and cloud power. It works well on devices like the Mac M1, making it accessible to a broad range of learners and developers.

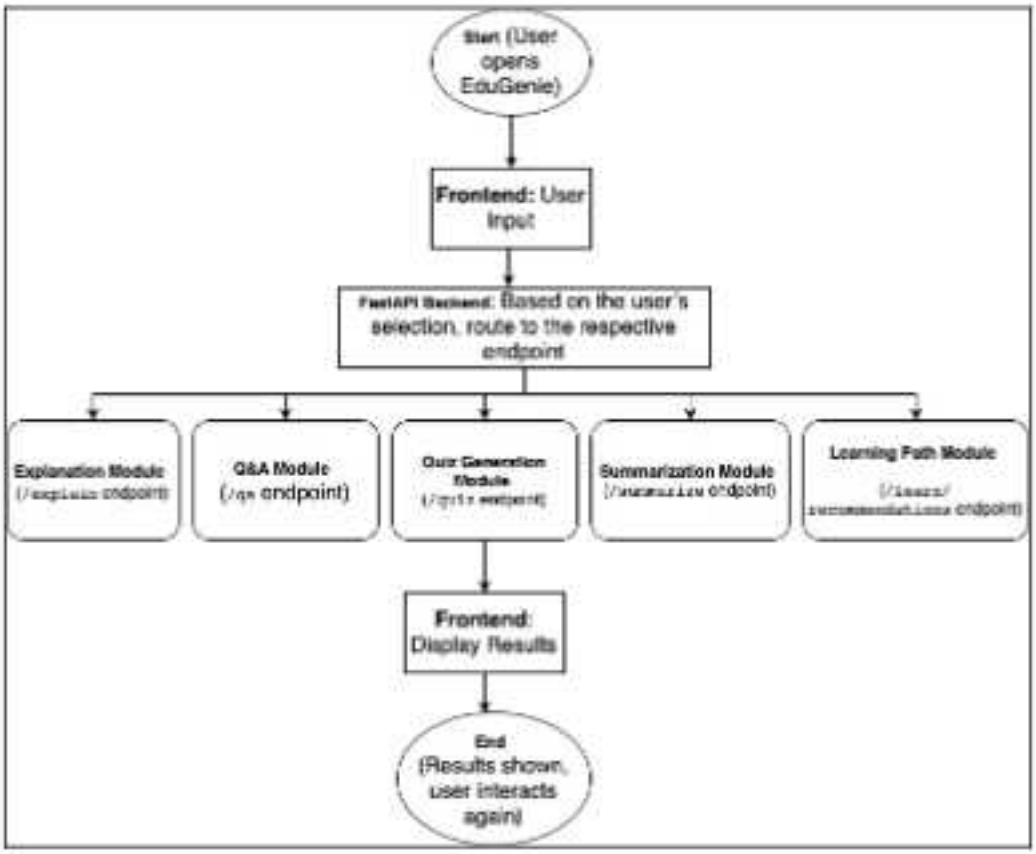
Scenarios

Scenario 1: A student wants to know about oceans and rivers uses EduGenie to ask "Which is the largest ocean?"

Scenario 2: A student wants to know the level of her understanding of "The Pythagoras Theorem" and clicks "Generate Quiz."

Scenario 3: A learner exploring SQL requests a learning path which is a structured plan with beginner to advanced topics, timelines, and suggestions.

ARCHITECTURAL DIAGRAM



Pre-requisites

1. Python 3.10+
   - Official Documentation: **Google AI for Developers**
   - Installation Guide: **Tom's Hardware**
   - Popular Tutorial: **Python Programming Tutorial - Full Course for Beginners**
   - Installation Steps:
     1. Download the latest Python 3.10+ installer for your operating system from the official website.
     2. Run the installer and ensure you check the box that says "Add Python to PATH".
     3. Follow the installation prompts to complete the setup.
     4. Verify the installation by opening a terminal or command prompt and typing python --version.
2. FastAPI Framework –
   - Official Documentation: FastAPI

- User Guide: FastAPI
- Popular Tutorial: FastAPI Crash Course

3. HTML & CSS – Basic templating used in /templates and /static
4. Google Gemini API Key

- Official Documentation: Google AI for Developers
- Setup Guide: GeeksforGeeks
- Popular Tutorial: How to Use Google Gemini API Key
- Setup Steps:
    1. Visit the Google AI Studio and sign in with your Google account.
    2. Create a new project and enable the Gemini API.
    3. Generate an API key and securely store it.

5. Uvicorn (ASGI Server)

- Official Documentation: PyPI
- Installation Steps:
    1. Install Uvicorn using pip: pip install uvicorn

6. Jinja2 (HTML Templating Engine)

- Official Documentation: Jinja2 Documentation
- Installation Steps:
    1. Install Jinja2 using pip: pip install jinja2

Project Workflow

MILESTONE 1: Model Selection and Architecture

Activity 1.1: Select AI Models

- **Gemini 1.5 Pro (via API):**
  Used for: Q&A, summarization, quiz generation, learning paths
  Benefits: Advanced reasoning, structured outputs, and cloud inference
- **LaMini-Flan-T5-783M (local):**
  Used for: Concept explanation
  Benefits: Instruction-tuned, lightweight, CPU-compatible

Folder Architecture:

EduGenie/
1. main.py # FastAPI app
2. explanation_module.py # Concept explanation logic
3. qna.py # Question answering
4. quiz_module.py # Quiz generation
5. summary_module.py # Summarization
6. learning_path.py # Learning recommendations
7. templates/index.html # HTML frontend
8. static/style.css # Styling
9. requirements.txt # Python dependencies



FIG. Showing the Folder Architecture

MILESTONE 2: Core Functionalities Development

Activity 2.1. : Module Implementation

1. Explanation Module

EduGenie utilizes the LaMini-Flan-T5 model, a lightweight yet powerful generative AI, to deliver educational content in a simplified and highly readable manner. This model is specifically fine-tuned to provide concise, context-aware responses that break down complex topics into easily understandable language. By focusing on clarity and brevity, EduGenie ensures that learners with minimal background knowledge or technical experience can grasp essential concepts without feeling overwhelmed. This makes it particularly valuable for beginners, school students, or self-learners seeking straightforward explanations. The integration of LaMini-Flan-T5 empowers EduGenie to act as a reliable and accessible study companion for foundational learning across subjects.

```
lanation_module.py
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

# Load the improved model for explanations
explain_tokenizer = AutoTokenizer.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")
explain_model = AutoModelForSeq2SeqLM.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")

def explain_topic(topic: str) -> str:
    input_text = f"Explain the concept of '{topic}' in a simple and clear way for a school student."
    inputs = explain_tokenizer(input_text, return_tensors="pt")

    outputs = explain_model.generate(
        **inputs,
        max_new_tokens=150,
        temperature=0.7,
        top_k=50,
        top_p=0.95,
        do_sample=True
    )

    explanation = explain_tokenizer.decode(outputs[0], skip_special_tokens=True)
    return explanation
```

Fig. Explanation module logic

2. QnA Module

EduGenie is powered by Gemini 1.5 Pro, enabling it to handle a wide range of general knowledge and academic question-answering tasks with precision. Its advanced understanding and contextual capabilities make it an effective tool for learners seeking accurate, AI-driven assistance across diverse educational topics and subjects.

```
def answer_question_with_gemini(question: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        response = model.generate_content(question)
        return response.text.strip()
    except Exception as e:
        return f"⚠️ Error in QnA: {e}"
```

Fig. Q/A Module

3. Quiz Module

EduGenie generates three multiple-choice questions (MCQs) from a given passage, each containing four carefully crafted options. It utilizes the Gemini model to comprehend the context and meaning of the passage, ensuring that the questions are relevant and the distractors are plausible. The final output is structured in JSON format for easy integration.

```
def clean_json_block(text):
    # Remove Markdown ```json code fences
    return re.sub(r"```(?:json)?\n(.*?)```", r"\1", text, flags=re.DOTALL).strip()

def generate_quiz(text: str) -> list:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")

        prompt = f"""
You are a quiz generator.

From the following passage, create 3 multiple-choice questions. Each question should include:
- A "question"
- A list of 4 "options"
- A correct "answer" that must exactly match one of the options.

Format your output as **valid JSON**, like this:
[
  {{
    "question": "What is ...?",
    "options": ["A", "B", "C", "D"],
    "answer": "A"
  }}
]

Passage:
{text}
"""
        response = model.generate_content(prompt)
        quiz_text = response.text.strip()

        # 🧼 Clean markdown code blocks if any
        cleaned_text = clean_json_block(quiz_text)
```

Fig. Quiz Module Logic

It sends a structured prompt to the model, expecting a valid JSON response with each question containing four options and a correct answer. To ensure proper parsing, it cleans any Markdown code blocks using the clean_json_blockfunction. The cleaned response is then parsed into a Python list. If an error occurs during generation or parsing, it returns a detailed error message for debugging.

4. Summary Module

This feature leverages Gemini's generative capabilities to summarize long paragraphs into concise, easy-to-understand versions, making it ideal for quick revision. It ensures the core information is retained while eliminating redundancy, helping learners absorb key points efficiently without losing clarity or context.

```python
def summarize_text(text: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        prompt = f"Summarize the following text in simple language:\n\n{text}"
        response = model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        return f"⚠ Error in Summary: {e}"
```

Fig. Summary Module

5. Learning Path Module

The get_learning_recommendations function uses Google Gemini to generate a personalized, structured learning path for any given topic. By crafting a detailed prompt, it instructs the AI to suggest beginner to advanced concepts, organized by difficulty, and supported with useful resources such as videos, articles, or books. It ensures adaptability to the learner's level. The function safely handles API responses and errors, providing either the generated text or a helpful error message if the response is invalid or incomplete.

```python
def get_learning_recommendations(topic):
    prompt = f"""
    You are an AI tutor. The student wants to learn about: {topic}.
    Suggest a structured and adaptive learning path including key topics, order of learning, and resources (
    Include beginner, intermediate, and advanced levels if needed.
    """
    try:
        response = model.generate_content(prompt)
        print("🧠 Gemini raw response:", response)

        if hasattr(response, "text"):
            return response.text
        elif hasattr(response, "parts") and response.parts:
            return response.parts[0].text
        else:
            return "❌ Could not extract content from Gemini response."
    except Exception as e:
        import traceback
        traceback.print_exc()  # 🔍 This gives full error trace
        return f"❌ Error occurred: {str(e)}"
```

Fig. Learning Path Module

Activity 2.2 : Backend API with FastAPI

- Defined RESTful endpoints for each module:

- /qa
- /explain
- /quiz
- /summarize
- /learn/recommendations

- Connected each endpoint to a respective module logic

```python
# Q&A - GET API using Gemini
@app.get("/qa")
async def answer_question(question: str = Query(...)):
    answer = answer_question_with_gemini(question)
    return {"answer": answer}

# Explanation - POST API
@app.post("/explain/")
async def explain_api(request: Request):
    data = await request.json()
    topic = data.get("topic")
    if not topic:
        return JSONResponse(content={"error": "Please provide a topic."}, status_code=400)
    explanation = explain_topic(topic)
    return {"topic": topic, "explanation": explanation}

# Summarization - POST API
@app.post("/summarize/")
async def summarize_api(request: Request):
    data = await request.json()
    text = data.get("text")
    if not text:
        return JSONResponse(content={"error": "Please provide text to summarize."}, status_code=400)
    summary = summarize_text(text)
    return {"summary": summary}
```

Fig. Endpoints of API

```python
# Quiz Generation - POST API
@app.post("/quiz")
async def quiz_api(request: Request):
    data = await request.json()
    text = data.get("text")
    if not text:
        return JSONResponse(content={"error": "Please provide text for quiz."}, status_code=400)
    quiz = generate_quiz(text)
    print("Generated quiz:", quiz)  # ✅ DEBUG
    return JSONResponse(content={"quiz": quiz})

# Learning Recommendations - GET API
@app.get("/learn/recommendations")
async def learning_recommendation_api(topic: str = Query(...)):
    recommendation = get_learning_recommendations(topic)
    return {"topic": topic, "recommendation": recommendation}
```

Fig. Endpoints of API

MILESTONE 3: Frontend Development

Activity 3.1: Build Web Interface

- HTML Form (index.html):
  - Task dropdown (Explain, QnA, Quiz, Summary, Recommend Path)
  - Text area for user input
  - Submit button
- CSS Styling (style.css):
  - Responsive design
  - Styled buttons, input areas, and result container

templates > ◇ index.html > ...

```
<html lang="en">
<head>
</head>
<body>
    <h1>Welcome to EduGenie 🎓</h1>
    <p>Your personal AI tutor for learning support!</p>

    <!-- QnA -->
    <form id="qaForm">
        <label><strong>Ask EduGenie a Question:</strong></label><br><br>
        <input type="text" id="question" placeholder="Why is the sky blue?" required>
        <button type="submit">Get Answer</button>
    </form>
    <div id="qaResult" class="output"></div>

    <!-- Explanation -->
    <form id="explainForm">
        <label><strong>Need an Explanation?</strong></label><br><br>
        <input type="text" id="topic" placeholder="Photosynthesis" required>
        <button type="submit">Explain</button>
    </form>
    <div id="explanationResult" class="output"></div>

    <!-- Summary -->
    <form id="summaryForm">
        <label><strong>Summarize a Paragraph:</strong></label><br><br>
        <input type="text" id="summaryText" placeholder="Paste long content to summarize" required>
        <button type="submit">Summarize</button>
    </form>
    <div id="summaryResult" class="output"></div>

    <!-- Quiz -->
    <form id="quizForm">
        <label><strong>Generate a Quiz:</strong></label><br><br>
        <input type="text" id="quizText" placeholder="Solar System" required>
        <button type="submit">Generate Quiz</button>
    </form>
    <div id="quizResult" class="output"></div>
```

Fig. Showing Frontend Intergration

Activity 3.2: Live Integration

- Form submission sends POST request to FastAPI backend
- Result appears below input box in real-time

MILESTONE 4: Deployment

- **Activity 4.1:** Run Locally
  Run locally with uvicorn main:app --reload
  Navigate to: http://127.0.0.1:8000



Fig. Showing the Application can be accessed

- **Activity 4.2:** Functional Testing
  Ask a question
  Get an explanation
  Generate a quiz
  Summarize content
  Get a personalized learning plan

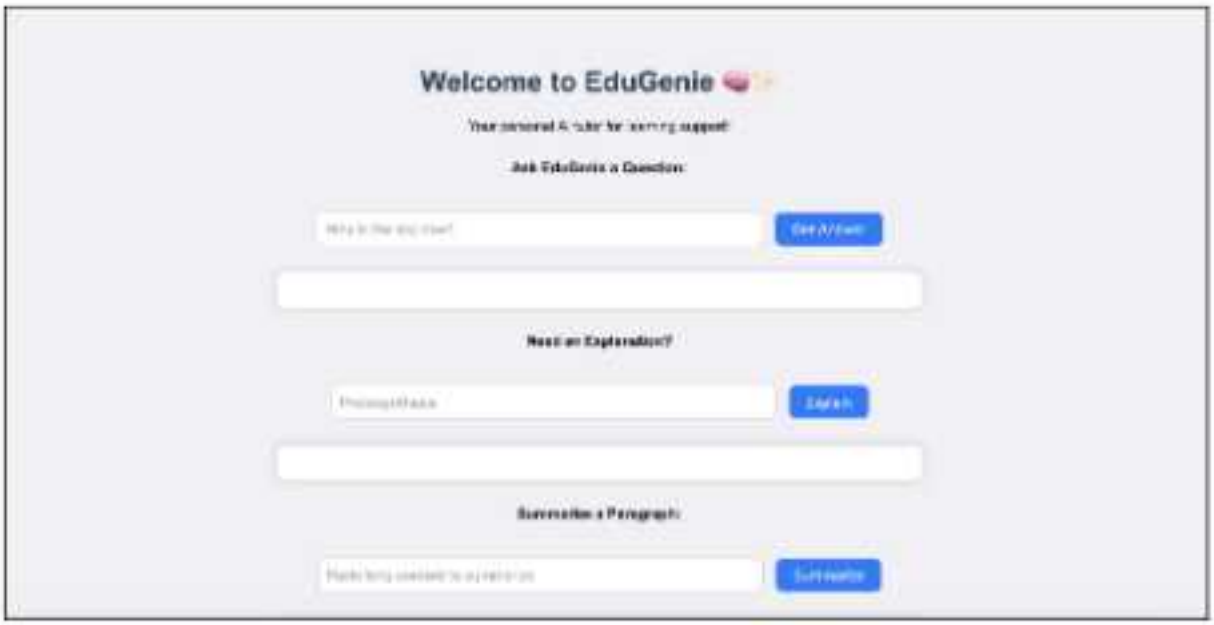


Fig. EDUGENIE



Fig. EDUGENIE

Exploring EduGenie:

a. Asking questions:



b. Explanation of any topic:





c. Summarising long paragraphs:



d. Generating quizzes:

It generates three questions with 4 options each. It corrects the answer if chosen wrong.







e. Guiding with learning recommendations from beginners level to advanced level



```
complex topics.  It's designed to be adaptive; feel free to adjust the
pace and delve deeper into areas that particularly interest you.

**I. Beginner Level: Building a Foundation**

**(Estimated Time: 1-2 weeks)**

* **Key Topics:**
    * What is a Database and SQL? (Relational Model, DBMS)
    * Basic Syntax (SELECT, FROM, WHERE)
    * Data Types (INT, VARCHAR, DATE, etc.)
    * Filtering Data (WHERE clause with comparison operators, logical
operators: AND, OR, NOT)
    * Ordering Results (ORDER BY)
    * Limiting Results (LIMIT/OFFSET or TOP)
    * Basic Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)
    * Introduction to Tables and Columns

* **Resources:**
    * **Interactive Tutorials:**
        * SQLZoo: Excellent for hands-on practice with different SQL
flavors.
        * Khan Academy's SQL course: Offers a good introductory
overview.
        * Codecademy's Learn SQL: Interactive lessons with practical
exercises.
    * **Videos:**
        * freeCodeCamp's SQL Tutorial: Comprehensive and beginner-
friendly.


**II. Intermediate Level: Working with Multiple Tables**

**(Estimated Time: 2-3 weeks)**

**II. Intermediate Level: Working with Multiple Tables**

**(Estimated Time: 2-3 weeks)**

* **Key Topics:**
    * Joining Tables (INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER
JOIN)
    * Subqueries (Nested queries)
    * Grouping Data (GROUP BY)
    * Filtering Groups (HAVING)
    * Set Operations (UNION, INTERSECT, EXCEPT)
    * Working with Strings (string concatenation, substrings, LIKE
operator)
    * Working with Dates and Times (date formatting, date arithmetic)
    * Views (Creating and using views)

* **Resources:**
    * **Books:**
        * "SQL Queries for Mere Mortals" by Michael J. Hernandez: A
great resource for understanding practical SQL querying.
    * **Online Documentation:** Refer to the documentation for your
specific SQL database (e.g., MySQL, PostgreSQL, SQL Server).
    * **Practice Platforms:** LeetCode, HackerRank, and similar
platforms offer SQL challenges to reinforce your skills.

**III. Advanced Level: Mastering Database Management**

**(Estimated Time: 3-4 weeks and beyond)**

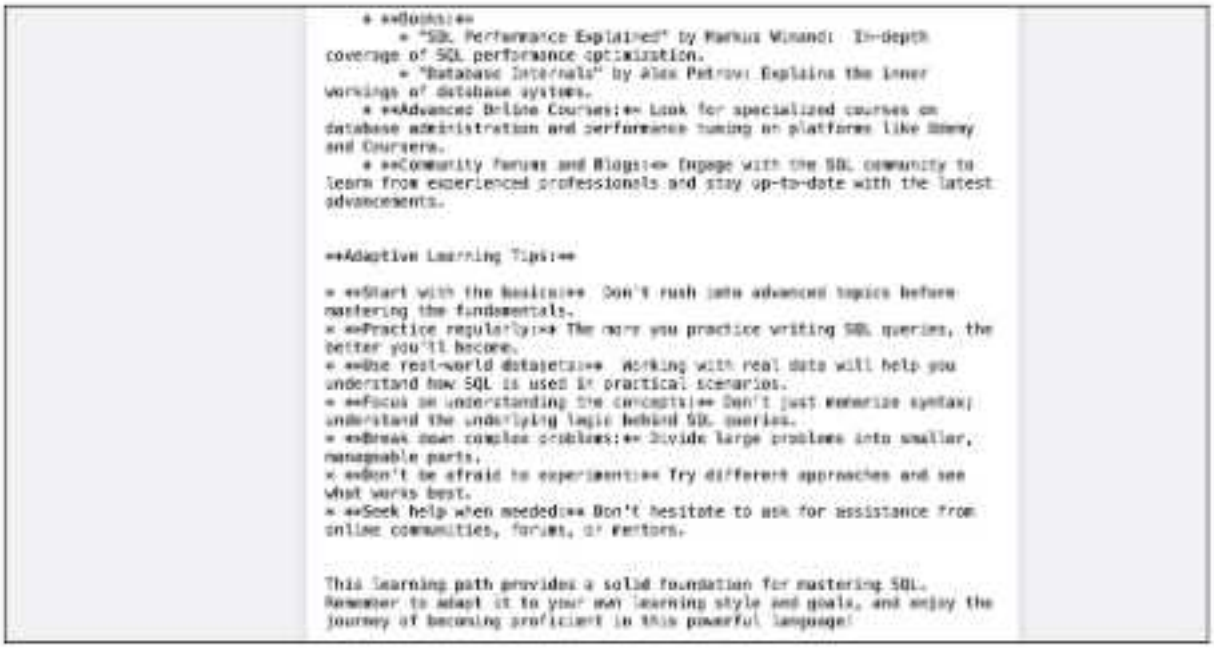
* **Key Topics:**
    * Stored Procedures and Functions
    * Triggers
    * Indexes and Performance Tuning
    * Transactions and Concurrency Control
    * Database Design (Normalization)
```

reaching a broader global audience. A mobile application is also envisioned to make learning available anytime, anywhere, with offline features for uninterrupted access.

Furthermore, future iterations of EduGenie could incorporate progress tracking dashboards, allowing users to visualize their learning journey through metrics and insights. The addition of gamified elements such as badges, learning streaks, and challenge-based assessments can boost engagement and motivation. Adaptive learning paths powered by data analytics can guide users more intelligently through content based on their strengths and weaknesses.

From a collaboration perspective, EduGenie can evolve to support group study sessions, teacher or parent dashboards, and integration with Learning Management Systems (LMS) such as Moodle or Google Classroom. Input recognition for images and PDF files can expand how students interact with content, enabling snapshot-based doubt solving and resource summarization. Real-time sync and smart notifications will also improve continuity and interactivity.

In conclusion, EduGenie is not just an AI tool—it's a foundation for the future of personalized, inclusive, and intelligent education. By bridging gaps in access, comprehension, and adaptability, it redefines the learning experience for digital natives and underserved communities alike. As it continues to evolve, EduGenie is poised to become a comprehensive learning companion for the modern era—smart, scalable, and learner-first.



It provides along with details of resources from where to learn and stepwise guidance through-out.

Conclusion:

EduGenie has been developed as a robust and accessible AI-powered educational assistant that seamlessly integrates cloud-based intelligence with an intuitive, low-footprint web interface. Designed to support self-learners, educational institutions, and content platforms, EduGenie democratizes learning by simplifying complex concepts, providing instant question-and-answer interactions, generating personalized quizzes, offering detailed summaries, and tailoring smart learning paths based on individual needs. Its lightweight infrastructure ensures that even users with minimal hardware resources can access high-quality, personalized education without barriers.

Throughout the development of EduGenie, the project achieved key milestones in AI integration, user experience design, and content adaptability. Leveraging advanced generative AI, it transforms static learning into a dynamic and interactive process. The platform's simplicity and clarity cater to learners of all levels—especially those who may find traditional educational content overwhelming or inaccessible. The modularity of the platform further allows for easy future upgrades, while its cloud-based backend ensures scalability and data-driven intelligence.

However, the development journey came with its own set of challenges. Ensuring accuracy in AI-generated responses, maintaining performance across devices, and balancing personlization with general usability were significant hurdles. Overcoming these required continuous testing, fine-tuning AI prompt structures, optimizing the UI for clarity, and addressing data privacy concerns. Yet, these challenges provided valuable learning outcomes—especially in areas like prompt engineering, cloud integration, user-centric design, and scalable architecture.

Looking ahead, EduGenie holds immense potential for enhancement and expansion. One promising direction involves the integration of voice-based interaction, enabling users to learn hands-free using spoken commands and queries—a particularly useful feature for accessibility and multitasking. Multilingual support is another key frontier, helping bridge language barriers and